

Smart Agri Advisor

Implementation Guide - Tomato Disease Detection, Soil Moisture Advisor and Hyperlocal Weather Alerts (English + Tamil)

1. Project Overview

This project is a bilingual (English/Tamil) web application for small and rural farmers. It has three connected modules: a CNN-based tomato leaf disease detector, a soil moisture and irrigation advisor driven by a Random Forest model and live weather data, and a hyperlocal weather alert screen with simple charts. All advice can also be played back as voice audio for farmers who are not comfortable reading.

1.1 How the modules connect

When a farmer uploads a diseased leaf photo, the CNN identifies the disease. The soil moisture module then checks whether poor irrigation may be a contributing factor, using rainfall and temperature pulled automatically from the weather API for the farmer's location - so the farmer does not need to enter rainfall manually or own a rain gauge.

2. Project Folder Structure

smart_agri_project/	
app.py	Main Flask application (all routes)
requirements.txt	Python dependencies
modules/	
disease_predictor.py	Loads CNN model, predicts disease
soil_moisture_predictor.py	Loads Random Forest model
weather_api.py	OpenWeatherMap integration + mock fallback
voice_advisor.py	Text-to-speech (English/Tamil)
translations.py	All EN/TA text strings
model_training/	
train_disease_model.py	CNN training script (PlantVillage)
train_soil_moisture_model.py	Random Forest training script
data/	
generate_soil_moisture_dataset.py	Synthetic dataset generator
templates/	HTML pages (Jinja2)
static/	
css/style.css	Nature-themed responsive styling
js/	disease.js, soil.js, weather.js, lang.js
audio/	Generated voice advice mp3 files
uploads/	Temporary storage for uploaded leaf photos
docs/	This guide

3. Setup Steps

- 1 Install Python 3.10 or newer.
- 2 Create a virtual environment: `python -m venv venv`, then activate it.
- 3 Install dependencies: `pip install -r requirements.txt`
- 4 Download the PlantVillage dataset from Kaggle and keep only the Tomato____* folders.

- 5 Set DATASET_DIR in model_training/train_disease_model.py to that folder's path.
- 6 Run: `python model_training/train_disease_model.py` (produces disease_model.h5 and class_indices.json)
- 7 Copy disease_model.h5 and class_indices.json into the modules/ folder.
- 8 Run: `python data/generate_soil_moisture_dataset.py` (creates the synthetic soil dataset)
- 9 Run: `python model_training/train_soil_moisture_model.py` (produces soil_moisture_model.pkl)
- 10 Copy soil_moisture_model.pkl and soil_moisture_encoders.pkl into the modules/ folder.
- 11 Create a free OpenWeatherMap account and get an API key.
- 12 Set the environment variable OPENWEATHER_API_KEY to that key (optional - a mock forecast is used automatically if this is skipped, so the demo still works).
- 13 Run the app: `python app.py`, then open `http://127.0.0.1:5000` in a browser.

4. Module 1: Tomato Leaf Disease Detector (CNN)

Algorithm: MobileNetV2 transfer learning. MobileNetV2 is chosen because it is small (about 14 MB), trains quickly even without a GPU, and is well suited to the texture/colour-based patterns that separate healthy leaves from diseased ones.

Workflow: Farmer photo -> CNN inference -> Disease class + confidence -> Localized advice text -> Optional voice playback.

Dataset: PlantVillage (Kaggle) - Tomato subset, 10 classes including healthy, early blight, late blight, bacterial spot, and others.

5. Module 2: Soil Moisture and Irrigation Advisor

Algorithm: Random Forest Classifier, predicting a 3-class irrigation need (Low / Medium / High) from weather and field features. Random Forest is chosen because it needs only a small-to-medium dataset, handles mixed numeric and categorical inputs without heavy preprocessing, trains in seconds, and is easy to explain to judges as 'many small decision rules voting together.'

Dataset: since a real IoT soil-sensor dataset was not available, a synthetic dataset is generated using a simplified crop water-balance rule (a lighter version of the FAO-56 Penman-Monteith method used in real agronomy): $\text{irrigation need} = \text{crop water requirement} - \text{effective rainfall} - \text{soil-stored moisture}$. This is a defensible proxy dataset for a hackathon MVP. As a real-data alternative, the Kaggle datasets 'Soil Moisture Dataset' (IoT sensor logs) or 'Crop Recommendation Dataset' can be adapted instead.

Input features: temperature, humidity, rainfall in the last 3 days (fetched automatically from the weather API using the farmer's location - no physical rain gauge needed), days since last irrigation, soil type, and crop growth stage.

Upgrade path: if a real low-cost capacitive soil moisture sensor is added later, its live reading can be added as one more input feature and the model retrained - no architecture change is needed.

6. Module 3: Hyperlocal Weather Alerts

Uses the OpenWeatherMap free forecast API to get a 5-day outlook for the farmer's exact location. Rain, heat, and frost thresholds convert raw numbers into simple alert flags. A bar-and-line chart (rainfall bars + temperature line) is rendered with Chart.js so farmers can see trends visually, not just read numbers. If there is no internet connection or API key, the module automatically falls back to a realistic mock forecast so the demo never breaks on stage.

7. Voice Advisory (Tamil and English)

Advice text is converted to speech using gTTS (Google Text-to-Speech), which supports both Tamil and English without needing an API key. Audio files are cached by content hash so the same advice is not regenerated twice. Voice INPUT (farmer asking a question by speaking) is listed as a stretch goal using the browser's built-in Web Speech API, since true Tamil speech recognition needs a heavier model such as Google Cloud Speech-to-Text or an on-device Vosk Tamil model.

8. Rural Constraint Handling

- **Connectivity:** the weather module falls back to a realistic mock forecast if the internet or API is unavailable, so the demo and core UI still function offline.
- **Literacy:** every screen has a Listen button that plays the advice as speech, and icons (camera, water drop, cloud) are used alongside text throughout.
- **Language:** a single toggle button switches the entire interface between English and Tamil instantly, stored per-session on the server.
- **Device requirements:** works on any basic smartphone browser - no native app install needed, and the camera capture attribute lets farmers take a photo directly.
- **Ease of use:** no manual sensor setup is required for the MVP - weather-based features replace the need for a physical soil-moisture sensor.

9. Realistic Hardware Cost Estimate

The MVP as described needs no special hardware - it runs on any basic smartphone with a camera and a browser, so hardware cost for the software demo is effectively zero beyond the farmer's existing phone. If a future version adds a physical soil-moisture sensor for more accurate readings than the weather-based estimate, an approximate low-cost bill of materials is:

Component	Approx. cost (INR)
Capacitive soil moisture sensor	150 - 250
Microcontroller (ESP8266 / ESP32)	250 - 450
Battery + solar trickle charger	300 - 500
Enclosure + wiring	100 - 150

Estimated total per unit	800 - 1,350

This avoids expensive equipment such as drones or laboratory sensors, keeping the solution realistic for small and rural farmers.

10. Future Work

- Add real IoT soil-moisture sensor data once deployed in the field, and retrain the model.
- Add Tamil voice INPUT using an on-device speech recognition model.

- Add SMS-based alerts for farmers without a smartphone or data plan.
- Expand disease detection beyond tomato to other crops using the same PlantVillage dataset.